FPT UNIVERSITY

FPT UNIVERSITY
HO CHI MINH CITY CAMPUS

FACULTY OF IC DESIGN AND DIGITAL AUTOMOTIVE ENGINEERING

PROJECT REPORT

FPGA IMPLEMENTATION OF A SECDED–BPSK COMMUNICATION PIPELINE WITH DETERMINISTIC ERROR AND NOISE INJECTION

Supervisor: Phạm Thế Vinh

No.	Student Name	Student ID
1	Nguyễn Trí Tuệ	SE205019
2	Lê Nhất Nam	SE205267
3	Trần Lê Tiến Đạt	SE205121
4	Hà Gia Bảo	SE205134
5	Tống Trần Đăng	SE204445

Implementation Period: 08 September 2026 – 01 November 2026

Ho Chi Minh City, 2026

ACKNOWLEDGEMENT

First of all, we would like to express our sincere gratitude to the lecturers of FPT University and, in particular, to the lecturers of the Faculty of IC Design and Digital Automotive Engineering for providing us with the fundamental knowledge and technical background required to complete this project.

We would especially like to thank our supervisor, Mr. Phạm Thế Vinh, for his valuable guidance, constructive feedback, and continuous support throughout the development of this project. His comments and suggestions during the design, implementation, verification, and report-preparation stages helped us improve our understanding of digital communication systems, error-control coding, FPGA-oriented RTL design, and hardware verification methodology.

We would also like to thank all members of our team for their cooperation and contributions throughout the project. The development of the RTL modules, integration of the complete communication pipeline, construction of the Python Golden Model and verification environment, analysis of synthesis and implementation results, and preparation of this report were made possible through effective teamwork and shared responsibility.

Although we have made every effort to complete the project carefully, our knowledge and practical experience are still developing, and the project may contain limitations or areas for further improvement. We sincerely welcome comments and suggestions from our lecturers so that we can continue to strengthen our technical knowledge, design skills, and research experience.

Finally, we would like to express our sincere appreciation to everyone who supported and encouraged us during the completion of this project.

ABSTRACT

Reliable digital communication requires both an effective modulation scheme and an error-control mechanism capable of protecting transmitted data against channel disturbances. In hardware-oriented communication systems, these functions must also be implemented with deterministic timing, limited hardware resources, and a verification methodology capable of ensuring bit-exact agreement between the algorithmic model and the RTL design.

This project focuses on the design and implementation of an FPGA-oriented digital communication pipeline combining an extended Hamming SECDED (8,4) code with Binary Phase Shift Keying (BPSK). The complete datapath consists of a Hamming encoder, an LFSR-based bit-error injection channel, a BPSK modulator, an LFSR-based additive-noise injector, a correlation-based BPSK demodulator, and a Hamming decoder. In addition, UART receiver, transmitter, and protocol-controller modules are integrated to form a complete top-level communication path.

A Python Golden Model is developed as the bit-exact reference for the RTL implementation. Deterministic test vectors are generated for every processing stage and are used by self-checking SystemVerilog testbenches. Each DSP module is first verified independently before being integrated into the complete `tx_rx_dsp_top` pipeline. Finally, a black-box end-to-end testbench verifies the complete path from the external UART receiver through the SECDED–BPSK processing chain and back to the UART transmitter.

The verification results demonstrate correct operation for the defined test dataset. The integrated DSP pipeline successfully completes 384 transactions covering 24,576 BPSK and noise samples, with 154,373 checks and no detected errors. The final UART end-to-end verification completes 96 requests with 577 checks and zero errors. Synthesis and implementation are also performed in Vivado for post-implementation analysis. The design satisfies the 50 MHz timing constraint with positive setup and hold slack, while using 165 LUTs and 213 flip-flops. The estimated total on-chip power reported by Vivado is 0.289 W.

The current project is completed at the RTL verification and post-implementation analysis level. Physical FPGA deployment and hardware measurements are outside the final project scope. Future work may extend the system toward FPGA-board validation, more realistic channel models, noise conditions capable of producing demodulation errors, and statistical evaluation such as bit-error-rate performance under different signal-to-noise conditions.

Contents

ACKNOWLEDGEMENT **1**

ABSTRACT **2**

1 PROJECT OVERVIEW **7**

1.1 Introduction 7
1.2 Project Objectives 7
1.3 Project Scope and Limitations 8
1.4 Development and Verification Methodology 8

2 THEORETICAL BACKGROUND **10**

2.1 Digital Communication System 10
2.2 Extended Hamming SECDED (8,4) 10
2.2.1 Overview 10
2.2.2 Codeword Mapping and Parity Generation 10
2.2.3 Syndrome and Error Classification 11
2.3 Binary Phase Shift Keying 13
2.3.1 BPSK Principle 13
2.3.2 Oversampling 13
2.4 Correlation-Based BPSK Demodulation 14
2.5 Linear Feedback Shift Register 15
2.6 Error and Noise Channel Model 16
2.6.1 Error Injection 16
2.6.2 Signed Additive Noise and Saturation 16
2.7 UART Communication 17
2.8 FPGA Fundamentals 18

3 SECDED–BPSK DSP ARCHITECTURE AND RTL DESIGN **19**

3.1 Overall System Architecture 19
3.2 DSP Top-Level Architecture 20
3.2.1 Top-Level Interface 20
3.2.2 Transaction and Handshake Behavior 20
3.3 Hamming SECDED Encoder 21

3.4 Deterministic Bit-Flip Channel 21
3.5 BPSK Modulator 22
3.6 LFSR Noise Injector 22
3.7 BPSK Demodulator 23
3.8 Hamming SECDED Decoder 23
3.9 Integrated DSP Pipeline 24
3.10 UART Controller and FPGA Top-Level Integration 24

4 VERIFICATION METHODOLOGY 26
4.1 Verification Strategy 26
4.2 Python Golden Model 27
4.3 Deterministic Reference Vector Generation 27
4.3.1 DSP Dataset 27
4.3.2 UART Dataset 28
4.4 Module-Level RTL Verification 28
4.5 Full DSP Pipeline Verification 29
4.6 UART End-to-End Verification 31
4.7 Verification Result Overview 32

5 BPSK DEMODULATOR DESIGN AND IMPLEMENTATION RESULTS 33
5.1 BPSK Demodulator RTL Design 33
5.1.1 Architecture and Interface 33
5.2 Demodulator Verification 34
5.2.1 Decision Margin and Noise-Model Limitation 34
5.3 Synthesis and Implementation Results 35
5.3.1 Resource Utilization 35
5.3.2 Timing Analysis 36
5.3.3 Routing and Power 36
5.4 Comparison with Representative FPGA Implementations 37

6 CONCLUSION AND FUTURE WORK 39
6.1 Conclusion 39
6.2 Project Limitations 39
6.3 Future Work 40

REFERENCES 41

List of Figures

1.1 Development and verification methodology of the proposed SECDED–BPSK communication system. 9

2.1 General processing flow of the proposed digital communication system. . . 10
2.2 Bit organization of the Extended Hamming SECDED $(8, 4)$ codeword. . . . 11
2.3 Two-stage XOR implementation of the parity equation $p_1 = d_0 \oplus d_1 \oplus d_3$. . 11
2.4 SECDED error-classification and correction flow. 12
2.5 Conceptual BPSK mapping between binary data and bipolar symbols. . . . 13
2.6 Eight-sample representation of each BPSK codeword bit. 14
2.7 Conceptual correlation-based BPSK demodulation process. 15
2.8 Conceptual eight-bit LFSR based on the selected feedback polynomial. . . 16
2.9 Conceptual signed-addition and saturation datapath. 17
2.10 UART 8-N-1 frame format. 18

3.1 Overall RTL architecture of the SECDED–BPSK communication system. . 19
3.2 Internal six-stage architecture of `tx_rx_dsp_top.v`. 20

4.1 Representative waveform of the integrated SECDED–BPSK DSP pipeline verification. 30
4.2 Representative UART end-to-end verification waveform of the complete `fpga_top` system. 31

5.1 Representative standalone simulation waveform of `tb_bpsk_demodulator.sv`. 34
5.2 Post-implementation resource utilization reported by Vivado. 35
5.3 Post-implementation timing summary at the 50 MHz target clock. 36
5.4 Post-implementation routing status reported by Vivado. 36
5.5 Vivado post-implementation on-chip power estimation. 37

List of Tables

2.1 SECDED error classification. 12
2.2 LFSR usage in the proposed system. 16

3.1 Interface of `tx_rx_dsp_top.v`. 20
3.2 Bit-error injection modes. 21

4.1 Module-level RTL verification status. 29
4.2 Integrated DSP verification result. 30
4.3 UART end-to-end verification result. 31
4.4 Summary of final verification stages. 32

5.1 Interface of `bpsk_demodulator.v`. 33
5.2 Summary of post-synthesis and post-implementation results. 35
5.3 Comparison with representative FPGA communication implementations. . 38

CHAPTER 1

PROJECT OVERVIEW

1.1 Introduction

Digital communication systems require mechanisms for reliable transmission in the presence of channel errors and noise. Error-control coding introduces redundancy for error detection and correction, while digital modulation provides a suitable representation for transmission [1].

This project combines Extended Hamming SECDED coding with Binary Phase Shift Keying (BPSK). Extended Hamming coding adds an overall parity bit to conventional Hamming coding, enabling Single-Error Correction and Double-Error Detection (SECDED) [2], [3]. BPSK represents binary data using two opposite signal polarities and is attractive for FPGA implementation because of its simple modulation and decision rules [1].

The proposed system implements a deterministic SECDED–BPSK pipeline in Verilog HDL. The processing chain includes SECDED encoding, controlled bit-error injection, BPSK modulation, LFSR-based additive-noise injection, correlation-based demodulation, and SECDED decoding. UART RX/TX and a protocol controller provide a byte-oriented external interface.

A bit-exact Python Golden Model is used to generate deterministic reference vectors for RTL verification. The verified design is then synthesized and implemented in Vivado for timing, utilization, routing, and estimated-power analysis.

1.2 Project Objectives

The project objectives are to:

- design and implement an Extended Hamming SECDED $(8, 4)$ encoder and decoder;
- implement deterministic single- and double-bit error injection;
- implement BPSK modulation and correlation-based demodulation using fixed-width signed arithmetic;
- implement deterministic LFSR-based additive noise with saturation;
- develop a Python Golden Model and deterministic reference-vector generator;
- verify standalone RTL blocks, the integrated DSP pipeline, and the UART end-to-end system using self-checking testbenches;

- evaluate the verified design using Vivado synthesis and implementation reports.

1.3 Project Scope and Limitations

The final project scope is defined by the following parameters:

- 4-bit payload and Extended Hamming SECDED $(8, 4)$ coding;
- BPSK mapping $0 \rightarrow +127$ and $1 \rightarrow -127$;
- eight samples per encoded bit and 64 samples per codeword;
- 8-bit signed BPSK/noisy samples;
- noise amplitudes $\{0, 4, 8, 16, 24, 32, 48, 64\}$;
- deterministic 8-bit LFSRs for error positions and noise polarity;
- UART at 115200 baud, 8-N-1, LSB-first;
- 50 MHz target system clock.

The noise source is deterministic rather than statistical AWGN, so the current design is intended for functional and bit-exact verification rather than BER-versus-SNR evaluation [1]. Physical FPGA-board programming, measured UART behavior, oscilloscope/logic-analyzer measurements, and measured board power are outside the final scope.

1.4 Development and Verification Methodology

The project follows a model-based RTL development flow. Algorithms are first defined in software, converted into deterministic reference vectors, implemented in synthesizable RTL, and then verified using self-checking testbenches before synthesis and implementation.

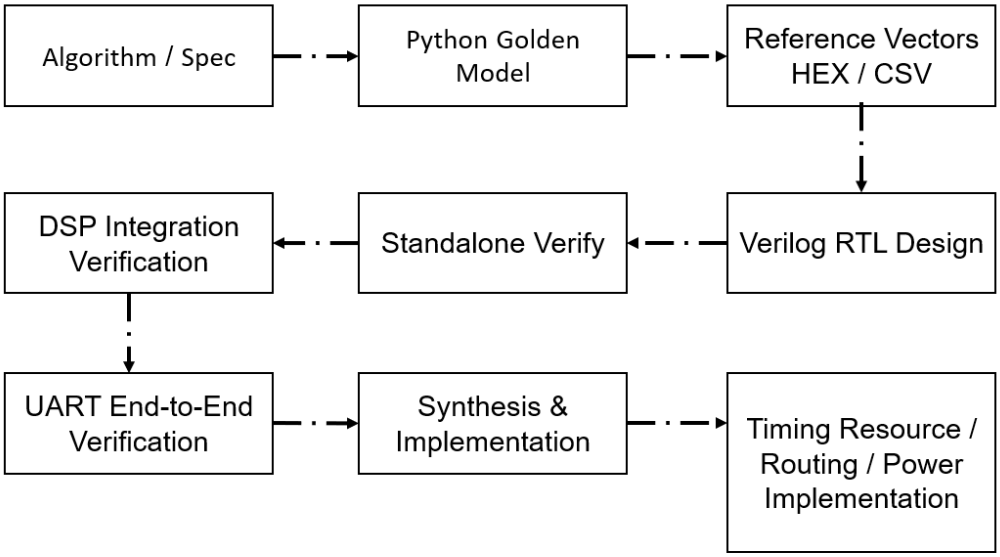


Figure 1.1. Development and verification methodology of the proposed SECDED–BPSK communication system.

The flow can be summarized as:

$$\begin{aligned} &\text{Specification} \rightarrow \text{Python Golden Model} \rightarrow \text{Reference Vectors} \\ &\quad \rightarrow \text{RTL} \rightarrow \text{Verification} \rightarrow \text{Vivado Analysis}. \end{aligned} \tag{1.1}$$

This approach separates algorithm correctness from RTL implementation details and provides deterministic evidence for each major verification stage.

CHAPTER 2

THEORETICAL BACKGROUND

This chapter summarizes the theoretical concepts required by the proposed SECDED–BPSK system. The discussion is intentionally limited to the principles needed to understand the RTL architecture presented in later chapters.

2.1 Digital Communication System

A digital communication system generally contains source/channel processing, modulation, a channel, demodulation, and decoding [1]. The proposed design follows the simplified flow shown in Figure 2.1.

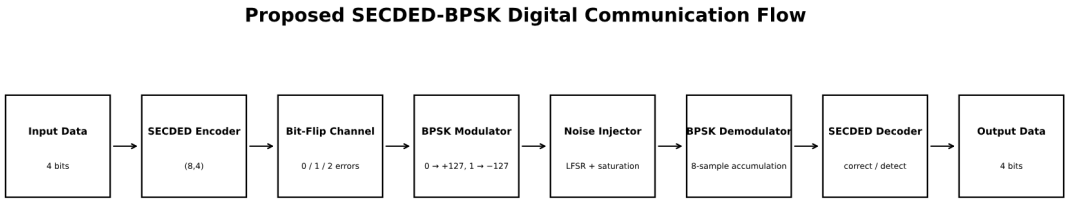


Figure 2.1. General processing flow of the proposed digital communication system.

2.2 Extended Hamming SECDED (8,4)

2.2.1 Overview

Hamming codes are linear block codes designed for error detection and correction [2]. Adding an overall parity bit extends the minimum distance and enables Single-Error Correction and Double-Error Detection (SECDED) [3]. In this project, four information bits are mapped to an eight-bit codeword.

2.2.2 Codeword Mapping and Parity Generation

The codeword organization is

$$\mathbf{c} = \{p_0, d_3, d_2, d_1, p_3, d_0, p_2, p_1\}. \tag{2.1}$$

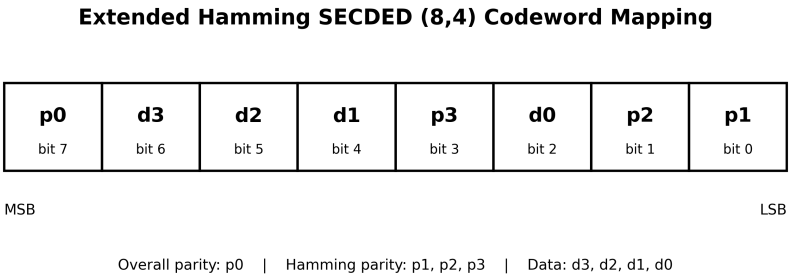


Figure 2.2. Bit organization of the Extended Hamming SECDED $(8, 4)$ codeword.

The Hamming parity bits are

$$p_1 = d_0 \oplus d_1 \oplus d_3, \tag{2.2}$$

$$p_2 = d_0 \oplus d_2 \oplus d_3, \tag{2.3}$$

$$p_3 = d_1 \oplus d_2 \oplus d_3. \tag{2.4}$$

The overall parity bit is

$$p_0 = p_1 \oplus p_2 \oplus d_0 \oplus p_3 \oplus d_1 \oplus d_2 \oplus d_3. \tag{2.5}$$

Because parity generation is XOR-based, the equations map naturally to combinational logic.

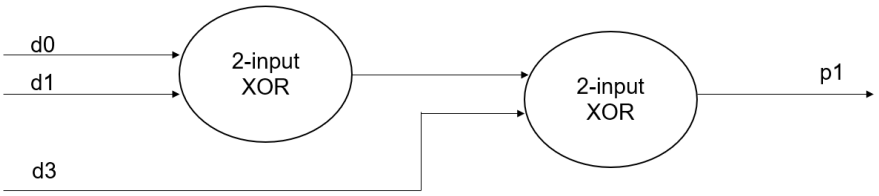


Figure 2.3. Two-stage XOR implementation of the parity equation $p_1 = d_0 \oplus d_1 \oplus d_3$.

2.2.3 Syndrome and Error Classification

The receiver recomputes the three Hamming checks

$$S_1 = p_1 \oplus d_0 \oplus d_1 \oplus d_3, \tag{2.6}$$

$$S_2 = p_2 \oplus d_0 \oplus d_2 \oplus d_3, \tag{2.7}$$

$$S_3 = p_3 \oplus d_1 \oplus d_2 \oplus d_3, \tag{2.8}$$

with

$$S = \{S_3, S_2, S_1\} \tag{2.9}$$

and overall parity

$$P = \bigoplus_{i=0}^{7} c_i. \tag{2.10}$$

The SECDED decision rule used in the project is summarized in Table 2.1 [3].

Table 2.1. SECDED error classification.

Syndrome	P	Status	Action
000	0	00	No correction
Non-zero	1	01	Correct bit $S-1$
000	1	01	Correct overall parity bit
Non-zero	0	10	Detect double-bit error

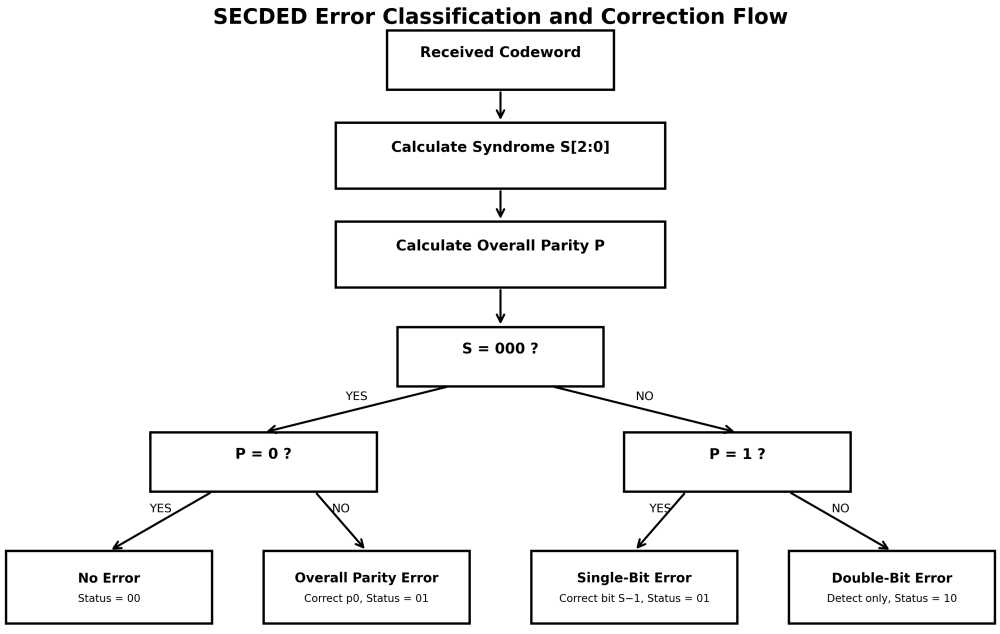


Figure 2.4. SECDED error-classification and correction flow.

2.3 Binary Phase Shift Keying

2.3.1 BPSK Principle

BPSK represents binary information using two carrier phases separated by 180° [1]. For the discrete digital-baseband representation used in this project, the mapping is simplified to

$$0 \rightarrow +127, \qquad 1 \rightarrow -127. \tag{2.11}$$

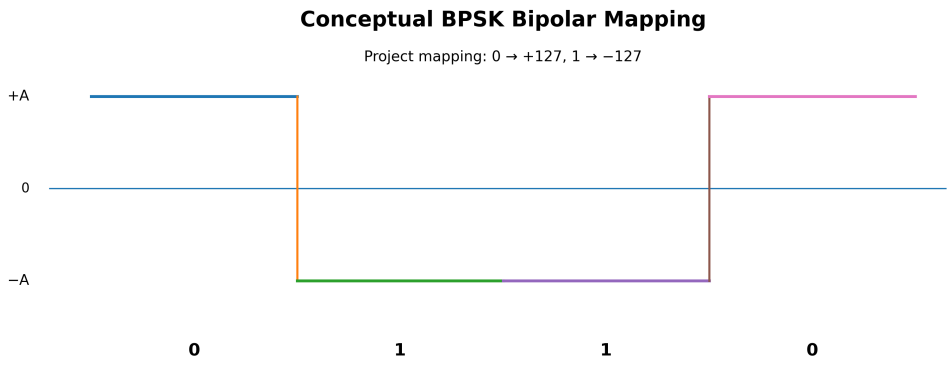


Figure 2.5. Conceptual BPSK mapping between binary data and bipolar symbols.

The values $+127$ and -127 fit directly in an 8-bit signed two's-complement sample.

2.3.2 Oversampling

Each encoded bit is represented by eight identical samples. One 8-bit SECDED codeword therefore produces

$$8 \text{ bits} \times 8 \text{ samples/bit} = 64 \text{ samples}. \tag{2.12}$$

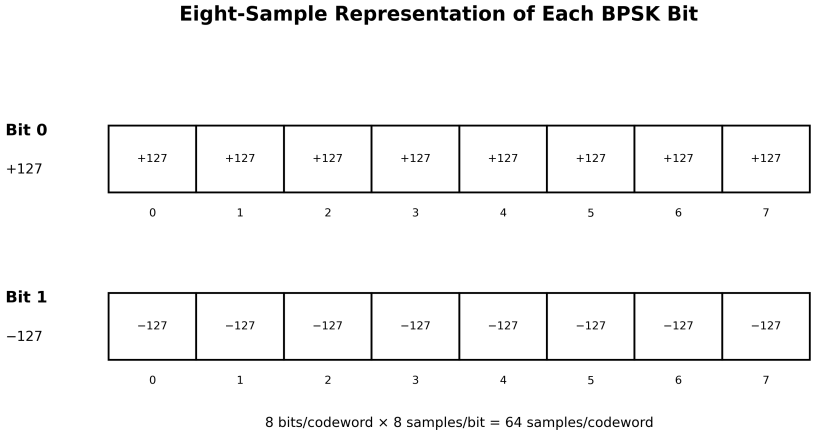


Figure 2.6. Eight-sample representation of each BPSK codeword bit.

2.4 Correlation-Based BPSK Demodulation

A coherent BPSK receiver can make a decision from the sign of a correlation statistic [1]. The digital implementation accumulates eight received samples:

$$A = \sum_{k=0}^{7} x[k]. \tag{2.13}$$

The decision rule is

$$\hat{b} = \begin{cases} 0, & A \geq 0, \\ 1, & A < 0. \end{cases} \tag{2.14}$$

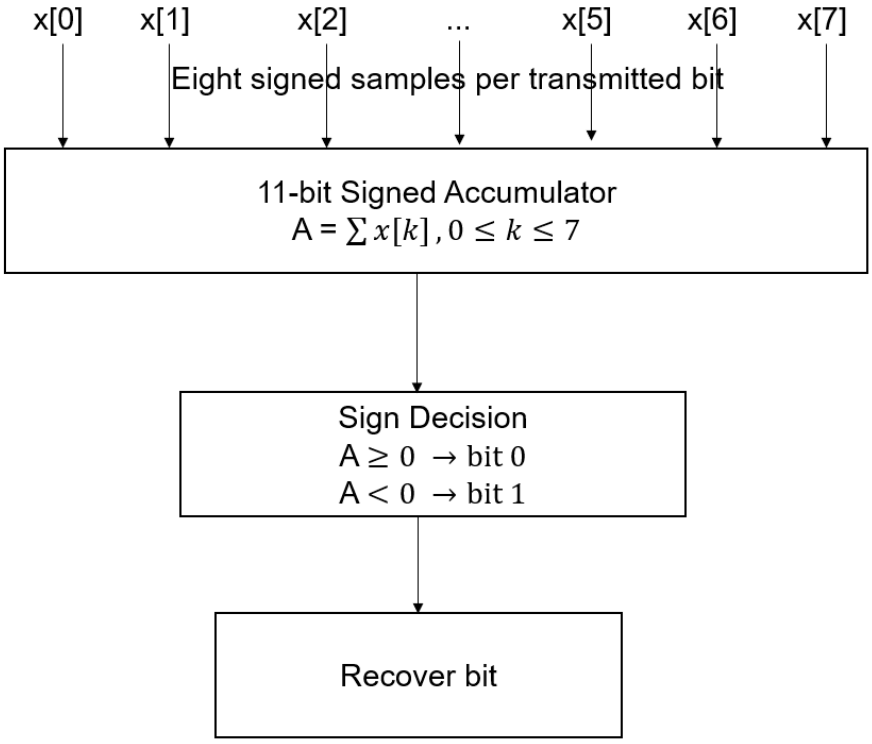


Figure 2.7. Conceptual correlation-based BPSK demodulation process.

For eight signed 8-bit samples,

$$-1024 \leq A \leq 1016. \tag{2.15}$$

Therefore, an 11-bit signed accumulator is sufficient.

2.5 Linear Feedback Shift Register

An LFSR is a shift register whose next state is generated by a linear XOR feedback function. LFSRs provide compact deterministic pseudo-random sequences and are widely used in digital systems [4].

The project uses the polynomial

$$x^8 + x^6 + x^5 + x^4 + 1, \tag{2.16}$$

which corresponds to

$$f = q_7 \oplus q_5 \oplus q_4 \oplus q_3. \tag{2.17}$$

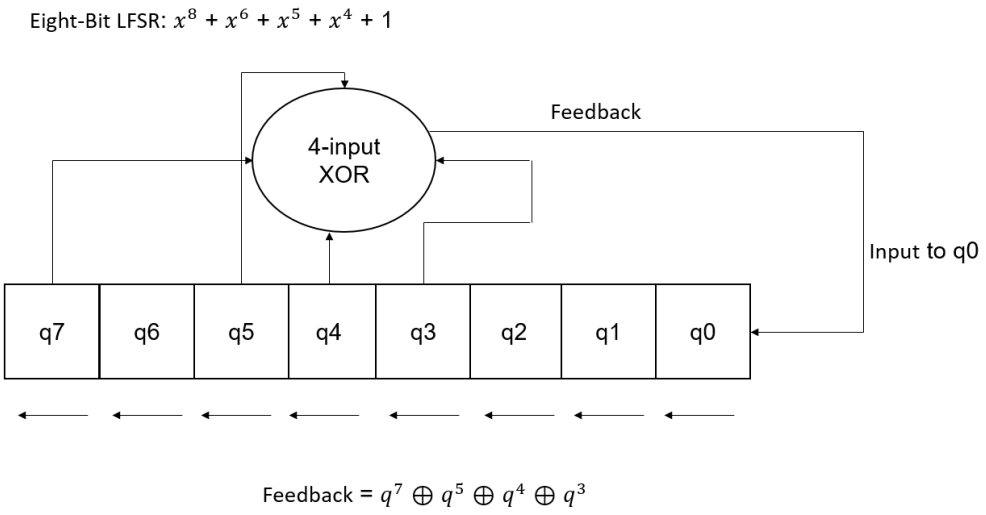


Figure 2.8. Conceptual eight-bit LFSR based on the selected feedback polynomial.

The two LFSRs used by the design are summarized below.

Table 2.2. LFSR usage in the proposed system.

LFSR	Seed	Update rate	Function
Error	`8'hA5`	Once/codeword	Error positions
Noise	`8'h01`	Once/sample	Noise sign

2.6 Error and Noise Channel Model

2.6.1 Error Injection

The deterministic bit-flip channel supports no error, one flipped bit, and two distinct flipped bits. The selected positions are derived from the error LFSR. This channel is intended for controlled verification rather than statistical channel modeling.

2.6.2 Signed Additive Noise and Saturation

The noise amplitude is selected from

$$\{0, 4, 8, 16, 24, 32, 48, 64\}, \tag{2.18}$$

and the noise LFSR selects its sign. The intermediate sum is

$$s = x + n. \tag{2.19}$$

Two signed 8-bit operands require a 9-bit intermediate representation:

$$-256 \leq s \leq 254. \tag{2.20}$$

The output is saturated to the valid signed 8-bit range:

$$y = \begin{cases} 127, & s > 127, \\ -128, & s < -128, \\ s, & \text{otherwise.} \end{cases} \tag{2.21}$$

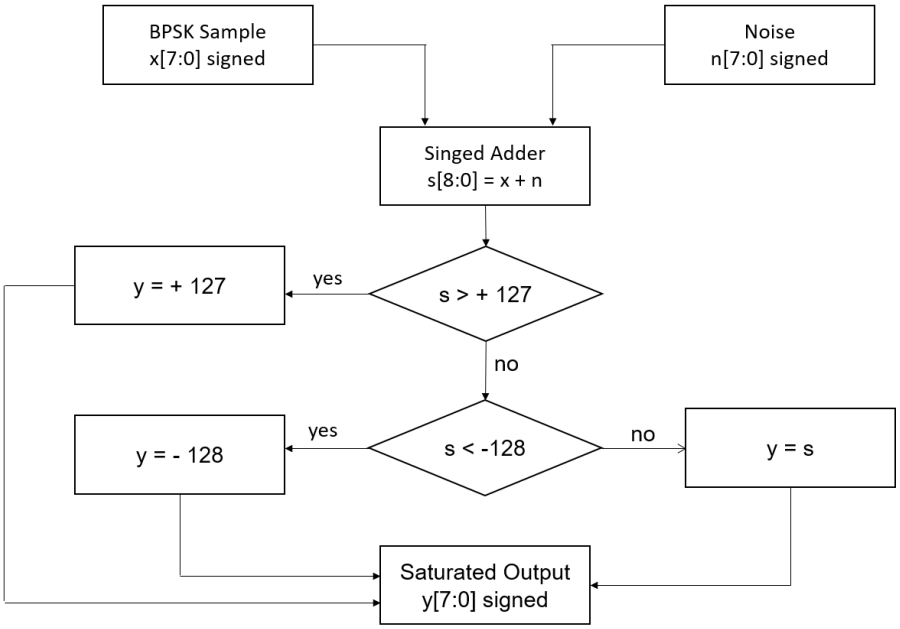


Figure 2.9. Conceptual signed-addition and saturation datapath.

The selected noise range is deterministic and does not represent statistical AWGN. Its measured effect on demodulation is discussed in Chapter 5.

2.7 UART Communication

UART converts parallel bytes to an asynchronous serial frame. The project uses 8-N-1 framing: one start bit, eight data bits, no parity, and one stop bit [5].

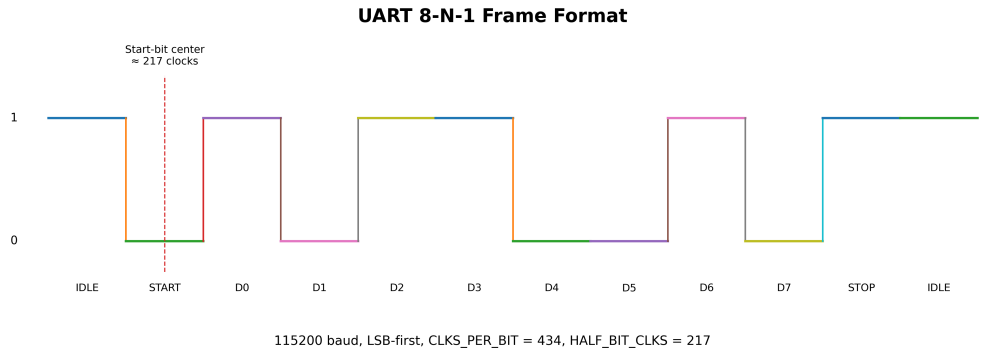


Figure 2.10. UART 8-N-1 frame format.

For a 50 MHz system clock and 115200 baud,

$$CLKS_PER_BIT = \frac{50,000,000}{115,200} \approx 434. \tag{2.22}$$

The receiver verifies the start bit near its center after approximately 217 system clocks and then samples subsequent bits once per bit period.

2.8 FPGA Fundamentals

An FPGA implements digital logic using configurable LUTs, registers, routing, clock resources, and optional dedicated blocks. Vivado synthesis maps RTL into a technology-specific netlist, while implementation performs optimization, placement, and routing [6], [7].

The proposed architecture mainly uses LUTs, flip-flops, counters, XOR logic, signed adders, and comparators. Fixed-width arithmetic also enables bit-exact comparison between the Python Golden Model and RTL implementation.

CHAPTER 3

SECDED–BPSK DSP ARCHITECTURE AND RTL DESIGN

This chapter presents the RTL architecture of the proposed SECDED–BPSK communication system. The theoretical principles were introduced in Chapter 2; therefore, the discussion here focuses on module organization, interfaces, control flow, and hardware implementation.

3.1 Overall System Architecture

The complete FPGA-oriented communication system consists of a UART front end and a six-stage DSP datapath. The external UART interface receives configuration and payload bytes, while the internal DSP pipeline performs channel coding, controlled error injection, modulation, deterministic noise injection, demodulation, and error correction.

The overall processing path is

UART RX → UART Controller → SECDED–BPSK DSP → UART Controller → UART TX. (3.1)

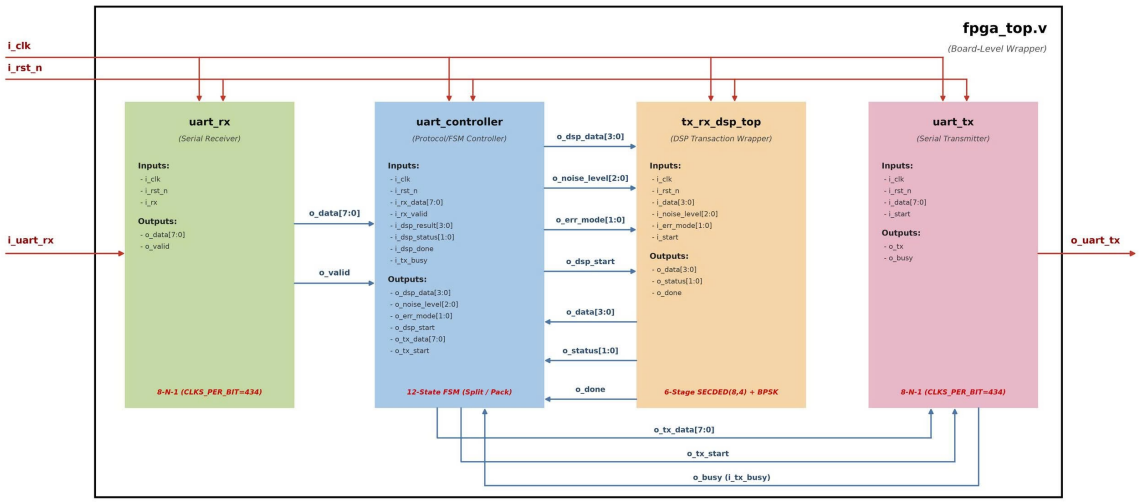


Figure 3.1. Overall RTL architecture of the SECDED–BPSK communication system.

The UART controller divides each received data byte into two four-bit DSP transactions. Each transaction is processed independently by `tx_rx_dsp_top.v`. The two decoded nib-

bles and their SECDED status values are then packed into UART response bytes.

3.2 DSP Top-Level Architecture

The module `tx_rx_dsp_top.v` integrates the six DSP stages shown in Figure 3.2.

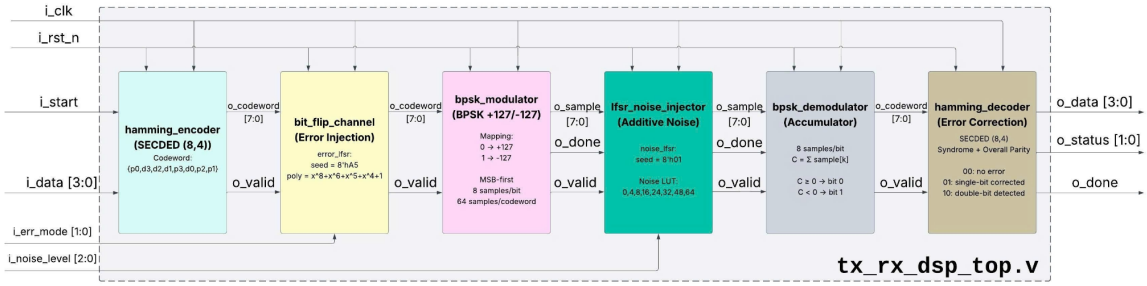


Figure 3.2. Internal six-stage architecture of `tx_rx_dsp_top.v`.

The processing sequence is

Encoder → Bit-Flip Channel → BPSK Modulator → Noise Injector → BPSK Demodulator → SECDE (3.2)

3.2.1 Top-Level Interface

Table 3.1. Interface of `tx_rx_dsp_top.v`.

Signal	Width	Direction	Description
`i_clk`	1	Input	System clock
`i_rst_n`	1	Input	Synchronous active-low reset
`i_data`	4	Input	Input payload nibble
`i_noise_level`	3	Input	Noise-amplitude selection
`i_err_mode`	2	Input	Bit-error injection mode
`i_start`	1	Input	Transaction request
`o_data`	4	Output	Decoded output nibble
`o_status`	2	Output	SECDED status
`o_done`	1	Output	Transaction-completion pulse

3.2.2 Transaction and Handshake Behavior

A transaction begins when `i_start` is asserted while the DSP pipeline is idle. The input payload, noise level, and error mode are captured for the transaction.

The RTL blocks use a simple valid/done protocol:

- `valid` indicates that the current-cycle data or sample is meaningful;
- stream-level `done` marks the final valid sample of a codeword stream;
- the top-level `o_done` pulse indicates completion of the complete DSP transaction.

Requests received while a processing block is busy are not accepted. All sequential logic uses the project-wide synchronous active-low reset convention.

3.3 Hamming SECDED Encoder

The module `hamming_encoder.v` converts one four-bit payload into one eight-bit Extended Hamming SECDED codeword.

The encoder implements the parity equations derived in Chapter 2 using combinational XOR logic. The resulting codeword is arranged as

$$\{p_0, d_3, d_2, d_1, p_3, d_0, p_2, p_1\}. \tag{3.3}$$

Because only XOR networks are required, the encoder does not use multipliers, memories, or iterative arithmetic. The encoded word is registered and forwarded to the deterministic bit-flip stage.

3.4 Deterministic Bit-Flip Channel

The module `bit_flip_channel.v` introduces controlled codeword errors before modulation. The supported modes are shown in Table 3.2.

Table 3.2. Bit-error injection modes.

Mode	Flipped Bits	Function
00	0	No error
01	1	Single-bit error
10	2	Double-bit error
11	0	Reserved; treated as no error

An 8-bit LFSR with seed `8'hA5` determines the error position. In single-error mode,

$$pos_1 = LFSR[2:0]. \tag{3.4}$$

In double-error mode, two positions are derived from different LFSR bit groups. If both positions are equal, the second position is adjusted so that two distinct bits are always corrupted.

The error LFSR advances once for every accepted codeword transaction, providing deterministic and reproducible error locations.

3.5 BPSK Modulator

The module `bpsk_modulator.v` converts the eight-bit SECDED codeword into a signed sample stream.

Each codeword bit is mapped to

$$0 \rightarrow +127, \qquad 1 \rightarrow -127, \tag{3.5}$$

and transmitted as eight identical signed 8-bit samples. Therefore, one codeword produces

$$8 \times 8 = 64 \tag{3.6}$$

samples.

The modulator processes the codeword in MSB-first order. Internal counters track the current codeword bit and sample index. The output `valid` signal is asserted for every generated sample, while the output `done` signal accompanies the sixty-fourth sample.

3.6 LFSR Noise Injector

The module `lfsr_noise_injector.v` adds deterministic signed noise to every valid BPSK sample.

A separate 8-bit LFSR with seed `8'h01` determines the noise polarity. The feedback function is

$$f = q_7 \oplus q_5 \oplus q_4 \oplus q_3. \tag{3.7}$$

The three-bit `i_noise_level` signal selects one of the amplitudes

$$\{0, 4, 8, 16, 24, 32, 48, 64\}. \tag{3.8}$$

The most significant bit of the current LFSR state selects positive or negative noise. The LFSR advances once per valid input sample.

The raw arithmetic result is evaluated using a wider signed intermediate value and then saturated to the valid 8-bit signed range:

$$-128 \leq o_sample \leq 127. \tag{3.9}$$

This prevents two's-complement wrap-around and guarantees deterministic agreement with the Python Golden Model.

3.7 BPSK Demodulator

The module `bpsk_demodulator.v` reconstructs the transmitted SECDED codeword from the noisy sample stream.

For every encoded bit, eight signed samples are accumulated using an 11-bit signed accumulator. After the eighth sample,

$$A \geq 0 \rightarrow 0, \qquad A < 0 \rightarrow 1. \tag{3.10}$$

Eight recovered bits are collected to reconstruct one SECDED codeword. The demodulator then asserts an output-valid pulse to the decoder.

The detailed datapath, control behavior, standalone verification, and hardware cost of this module are presented in Chapter 5.

3.8 Hamming SECDED Decoder

The module `hamming_decoder.v` evaluates the syndrome and overall parity of the demodulated codeword.

The decoder classifies the received word into four cases:

- no error;
- correctable single-bit error;
- overall-parity-bit error;
- detected double-bit error.

For a correctable single-bit error, the syndrome identifies the corrupted Hamming position and the corresponding bit is inverted. A detected double-bit error is reported but not corrected.

After correction, the original payload is extracted from codeword positions corresponding to

$$\{d_3, d_2, d_1, d_0\}. \tag{3.11}$$

The decoder outputs the recovered four-bit payload together with the two-bit SECDED status.

3.9 Integrated DSP Pipeline

The six modules are connected sequentially inside `tx_rx_dsp_top.v`. Word-level stages operate on complete SECDED codewords, while the BPSK path operates on a 64-sample stream.

The top-level module therefore bridges two processing domains:

$$\text{word domain} \rightarrow \text{sample-stream domain} \rightarrow \text{word domain}. \tag{3.12}$$

The encoder and bit-flip channel operate once per codeword. The modulator expands the word into 64 samples, the noise injector processes each sample individually, and the demodulator compresses the stream back into one eight-bit word for SECDED decoding.

The two LFSR states are persistent across transactions. They are reset only by the global reset and otherwise advance according to their stage-specific update rules. This behavior is reproduced exactly by the Python Golden Model and is verified in Chapter 4.

At the end of a transaction, the decoder result is transferred to `o_data` and `o_status`, and `o_done` is asserted for one clock cycle. This output forms the interface between the DSP datapath and the UART controller.

3.10 UART Controller and FPGA Top-Level Integration

The complete communication path is integrated in `fpga_top.v`, which connects the UART receiver, the protocol controller, the SECDED–BPSK DSP pipeline, and the UART transmitter.

The UART receiver converts the incoming serial stream into 8-bit data bytes and asserts a valid pulse when a complete frame has been received. These bytes are forwarded to `uart_controller.v`, which implements a 12-state finite-state machine responsible for protocol sequencing, DSP transaction control, and response generation.

Each received DATA byte is divided into two four-bit nibbles. The controller processes the upper and lower nibbles sequentially by applying the selected error mode and noise level to `tx_rx_dsp_top.v`. For each nibble, the controller asserts the DSP start signal and waits for the corresponding completion pulse before advancing to the next processing step.

After both nibble transactions have completed, the decoded values and their SECDED status information are packed into UART response bytes. The controller then supplies these bytes to `uart_tx.v` and initiates serial transmission.

The top-level integration therefore performs the following sequence:

$$\begin{aligned} &\text{Receive UART command} \rightarrow \text{Decode configuration/data} \\ &\rightarrow \text{Process upper nibble} \rightarrow \text{Process lower nibble} \\ &\rightarrow \text{Pack decoded data/status} \rightarrow \text{Transmit UART response.} \end{aligned} \tag{3.13}$$

The UART blocks operate at 115200 baud using 8-N-1 framing, while all internal control and DSP logic are driven by the common 50 MHz system clock. The UART transmitter busy signal is monitored by the controller to prevent a new response byte from being launched while a previous byte is still being transmitted.

This organization separates serial communication control from the DSP datapath. The `uart_controller` manages transaction-level sequencing, whereas `tx_rx_dsp_top` remains responsible only for one four-bit SECDED–BPSK processing transaction. The resulting structure corresponds to the overall architecture shown in Figure 3.1 and is verified end-to-end in Chapter 4.

CHAPTER 4

VERIFICATION METHODOLOGY

This chapter presents the verification methodology used to validate the SECDED–BPSK communication system. The objective is to establish bit-exact consistency between the software reference model and the RTL implementation at module, pipeline, and system levels.

4.1 Verification Strategy

The verification flow is based on a deterministic software reference model:

$$\text{Specification} \rightarrow \text{Python Golden Model} \rightarrow \text{Reference Vectors} \rightarrow \text{RTL} \rightarrow \text{Self-Checking Testbench}. \tag{4.1}$$

The same algorithmic rules are used by the Golden Model and the RTL design, including:

- SECDED encoding and decoding;
- deterministic error-position generation;
- BPSK symbol mapping;
- deterministic LFSR noise generation;
- signed saturation arithmetic;
- correlation-based BPSK demodulation;
- persistent error/noise LFSR state evolution.

Verification is performed at three levels:

1. standalone verification of each DSP processing block;
2. integrated verification of the complete six-stage DSP pipeline;
3. black-box UART end-to-end verification of the complete top-level system.

4.2 Python Golden Model

The Python Golden Model is the bit-exact algorithmic reference for the RTL design. For each transaction, it receives the input payload, error mode, and noise level, then reproduces the expected behavior of all DSP stages.

For one transaction, the model produces the following reference information:

- encoded SECDED codeword;
- corrupted codeword after deterministic bit-error injection;
- BPSK sample stream;
- noise sequence and saturated noisy samples;
- demodulated codeword;
- syndrome and corrected codeword;
- decoded four-bit payload and SECDED status;
- error-LFSR and noise-LFSR states before and after processing.

The use of persistent LFSR state is important because the expected result of one transaction depends on the state left by previous transactions. Therefore, verification checks both datapath values and deterministic state evolution.

4.3 Deterministic Reference Vector Generation

Reference vectors are exported from Python into HEX files and transaction records. These files are loaded directly by the self-checking SystemVerilog testbenches.

For the noise-processing stage, the generated reference data include the transmitted BPSK samples, signed noise values, saturated noisy samples, and the noise-LFSR states before and after each transaction. Representative files include `bpsk_tx.hex`, `noise_values.hex`, `bpsk_noisy.hex`, `noise_lfsr_before.hex`, and `noise_lfsr_after.hex`.

4.3.1 DSP Dataset

The integrated DSP dataset covers all 16 possible four-bit payload values, all eight supported noise levels, and the three useful error modes:

$$16 \times 8 \times 3 = 384 \tag{4.2}$$

DSP transactions.

Each transaction contains eight SECDED bits and eight BPSK samples per encoded bit:

$$8 \times 8 = 64 \tag{4.3}$$

samples per transaction.

Therefore, the full regression contains

$$384 \times 64 = 24{,}576 \tag{4.4}$$

BPSK/noise samples.

The generated reference-vector set contains stage-level outputs and persistent LFSR states so that internal behavior can be checked in addition to the final decoded result.

4.3.2 UART Dataset

The UART black-box regression uses

$$8 \times 3 \times 4 = 96 \tag{4.5}$$

request transactions, covering eight noise levels, three useful error modes, and four representative input-byte patterns.

Each input byte is processed as two four-bit DSP transactions. Therefore,

$$96 \times 2 = 192 \tag{4.6}$$

DSP transactions are executed during the complete UART regression.

4.4 Module-Level RTL Verification

Each DSP stage is first verified independently before integration. The standalone testbenches apply deterministic stimulus and compare the RTL outputs against the corresponding expected values.

Table 4.1. Module-level RTL verification status.

Module	Verification Method	Result	Known Checks
`hamming_encoder.v`	Self-checking standalone TB	PASS	69
`bit_flip_channel.v`	Self-checking standalone TB	PASS	1,537
`bpsk_modulator.v`	Self-checking standalone TB	PASS	–
`lfsr_noise_injector.v`	Self-checking standalone TB	PASS	–
`bpsk_demodulator.v`	Self-checking standalone TB	PASS	–
`hamming_decoder.v`	Self-checking standalone TB	PASS	1,954

The exact standalone check totals for the BPSK modulator, noise injector, and demodulator are not used as final project metrics; however, all three standalone regressions complete with zero detected errors.

Standalone verification isolates local implementation errors before full-pipeline integration. This is particularly useful for identifying problems such as incorrect bit ordering, LFSR update timing, signed arithmetic width, or valid/done timing.

For the LFSR noise injector, standalone verification specifically checks reset initialization, deterministic state progression, configurable noise-amplitude selection, signed-noise polarity, saturation behavior, and `valid`/`done` timing.

4.5 Full DSP Pipeline Verification

After all six processing stages pass standalone verification, they are integrated in `tx_rx_dsp_top.v`.

The integrated testbench checks both external outputs and major internal reference points. These include:

- SECDED encoded and corrupted codewords;
- error-LFSR state;
- BPSK samples;
- generated noise values;

- noise-LFSR state;
- saturated noisy samples;
- demodulated codeword;
- syndrome and corrected codeword;
- decoded payload and SECDED status;
- top-level completion behavior.

For the noise-processing stage, the integrated regression additionally compares the generated signed noise value, LFSR state progression, and saturated noisy BPSK sample against the corresponding Python reference vectors.

The final integrated regression result is summarized in Table 4.2.

Table 4.2. Integrated DSP verification result.

Metric	Result
DSP transactions	384
BPSK/noise samples	24,576
Self-checking comparisons	154,373
Detected errors	0
Final result	PASS

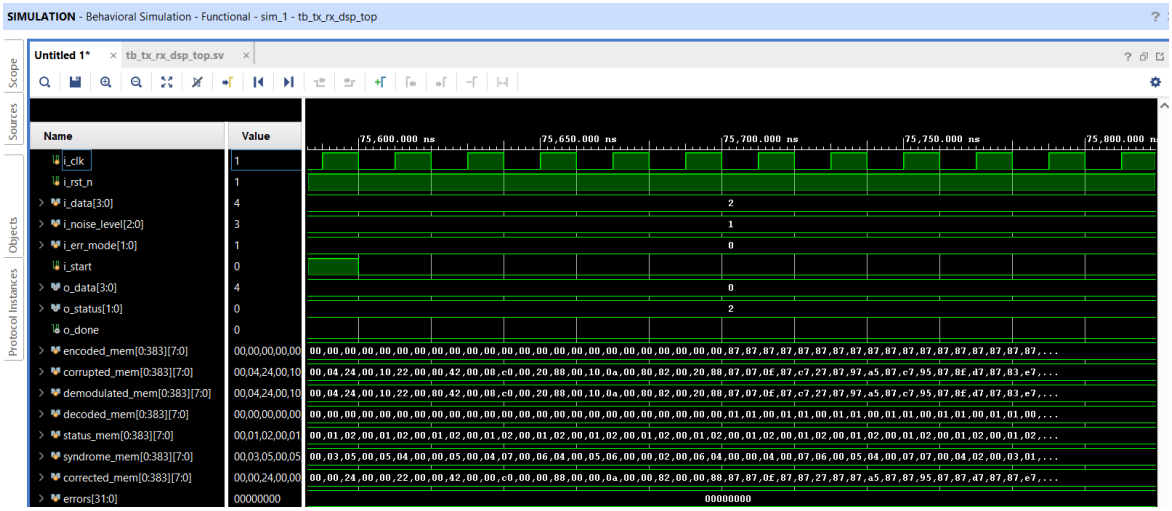


Figure 4.1. Representative waveform of the integrated SECDED–BPSK DSP pipeline verification.

The zero-error result confirms bit-exact agreement between the RTL pipeline and the stateful Python Golden Model for the complete deterministic dataset.

4.6 UART End-to-End Verification

The final verification stage treats `fpga_top` as a black box. The testbench drives serialized UART frames into `i_uart_rx` and monitors serialized response frames from `o_uart_tx`.

The complete signal path is

$$\text{UART RX} \rightarrow \text{UART Controller} \rightarrow \text{DSP Pipeline} \rightarrow \text{UART Controller} \rightarrow \text{UART TX}. \tag{4.7}$$

The testbench does not directly force internal DSP signals. Instead, it verifies the behavior visible at the external UART interface. This checks both serial communication and the complete internal processing chain.

The final UART regression result is shown in Table 4.3.

Table 4.3. UART end-to-end verification result.

Metric	Result
UART requests	96
Equivalent DSP transactions	192
Self-checking comparisons	577
Detected errors	0
Final result	PASS
Simulation completion time	898750 ns

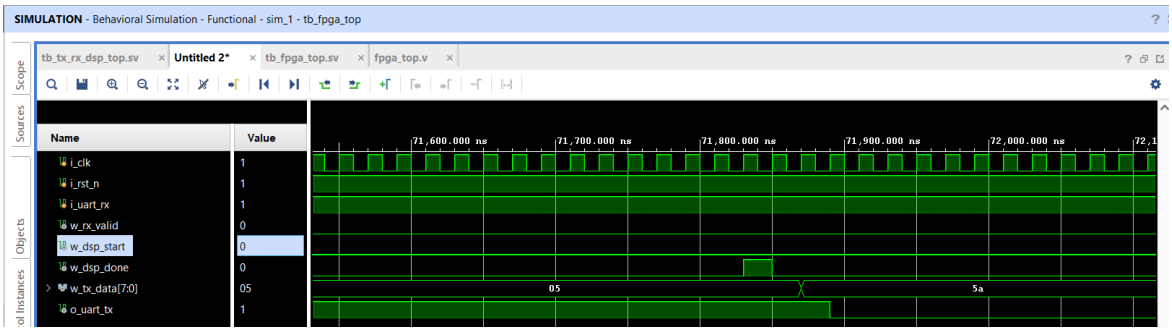


Figure 4.2. Representative UART end-to-end verification waveform of the complete `fpga_top` system.

This result confirms correct integration of UART framing, controller sequencing, nibble-level DSP transactions, response packing, and end-to-end request/response behavior.

4.7 Verification Result Overview

The complete verification process progresses from isolated RTL blocks to the integrated datapath and finally to the external serial interface.

Table 4.4. Summary of final verification stages.

Level	Scope	Transactions	Checks	Result
Module	Six standalone DSP stages	–	Module-specific	PASS
DSP Integration	Complete `tx_rx_dsp_top`	384	154,373	PASS
UART End-to-End	Complete `fpga_top`	96 requests	577	PASS

The verification results demonstrate deterministic agreement from the software reference model to the final UART-visible behavior. These verified RTL results are used as the basis for the synthesis and implementation analysis presented in Chapter 5.

CHAPTER 5

BPSK DEMODULATOR DESIGN AND IMPLEMENTATION RESULTS

This chapter presents the RTL implementation of the BPSK demodulator and the post-implementation results of the complete system. BPSK theory and the correlation decision rule were introduced in Chapter 2.

5.1 BPSK Demodulator RTL Design

5.1.1 Architecture and Interface

The demodulator accumulates eight signed samples for each encoded bit and applies the zero-threshold decision derived in Chapter 2. It is implemented using two 3-bit counters, one 11-bit signed accumulator, an 8-bit codeword register, and sign-decision logic.

Table 5.1. Interface of `bpsk_demodulator.v`.

Signal	Width	Direction	Description
`i_clk`	1	Input	System clock
`i_rst_n`	1	Input	Synchronous active-low reset
`i_sample`	8 signed	Input	Noisy BPSK sample
`i_valid`	1	Input	Current sample is valid
`i_done`	1	Input	Final valid sample of the frame
`o_codeword`	8	Output	Reconstructed SECDED codeword
`o_valid`	1	Output	One-clock codeword-valid pulse

For samples 0–6, the input is accumulated normally. On sample 7, the design evaluates

$$\texttt{acc_next} = \texttt{demod_acc} + \texttt{sample_ext} \tag{5.1}$$

so that the final sample contributes to the current-bit decision without an additional cycle. The signed 8-bit input is sign-extended before addition:

```
sample_ext = {{3{i_sample[7]}}, i_sample};
acc_next   = demod_acc + sample_ext;
```

A completed codeword is released only after the final sample of bit 7. The `o_valid` output is asserted for one clock.

5.2 Demodulator Verification

The standalone testbench verifies representative all-zero, all-one, alternating, and mixed codewords. Eleven codewords (704 samples) are processed with zero detected errors.

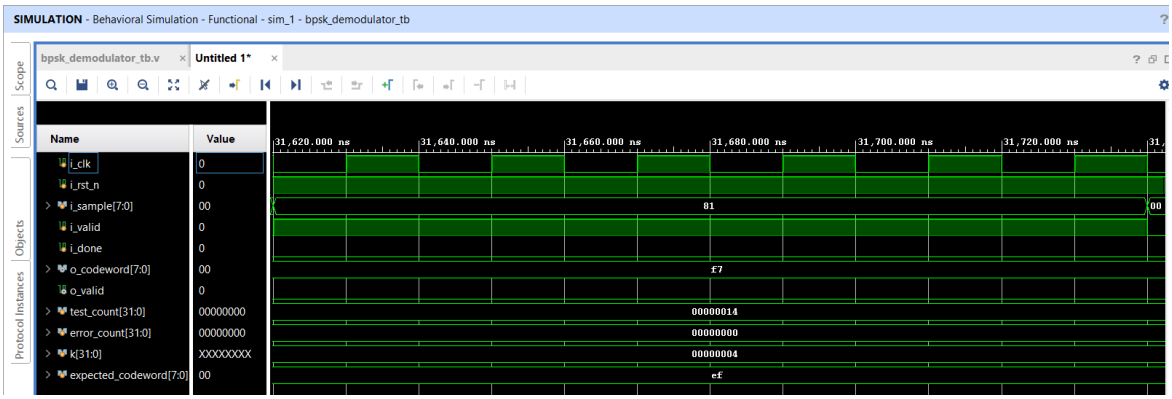


Figure 5.1. Representative standalone simulation waveform of `tb_bpsk_demodulator.sv`.

Full-system verification follows the methodology presented in Chapter 4. The demodulator passes standalone verification, and the complete DSP and UART regressions also complete with zero detected errors. Detailed dataset sizes and check counts are reported in Sections 4.5 and 4.6.

5.2.1 Decision Margin and Noise-Model Limitation

The maximum configured noise magnitude is 64. Therefore,

$$127 - 64 = 63 > 0, \qquad -127 + 64 = -63 < 0. \tag{5.2}$$

The configured noise cannot independently reverse the polarity of an individual BPSK sample. The theoretical minimum magnitude of an eight-sample correlation under this sign-preserving model is

$$|\Sigma| \geq 8 \times 63 = 504. \tag{5.3}$$

The minimum observed correlation magnitude in the generated dataset is 568. This confirms a substantial decision margin for the deterministic regression. However, because the configured noise model is sign-preserving and non-Gaussian, the result validates arithmetic and datapath correctness rather than BER-versus-SNR communication performance.

5.3 Synthesis and Implementation Results

The complete `fpga_top` design is synthesized and implemented in Vivado using a 50 MHz timing constraint. Synthesis maps the RTL into a technology-specific netlist, while implementation performs optimization, placement, and routing [6], [7]. The results in this section therefore represent post-synthesis and post-implementation tool reports, not measurements from a physical FPGA board.

Table 5.2. Summary of post-synthesis and post-implementation results.

Metric	Result
Post-synthesis LUTs	177
Post-implementation LUTs	165
Flip-flops	213
BRAM / URAM / DSP	0 / 0 / 0
Target clock	50 MHz (20 ns)
WNS / TNS	+17.949 ns / 0.000 ns
WHS / THS	+0.039 ns / 0.000 ns
Failed routes	0
Estimated on-chip power	0.289 W

5.3.1 Resource Utilization

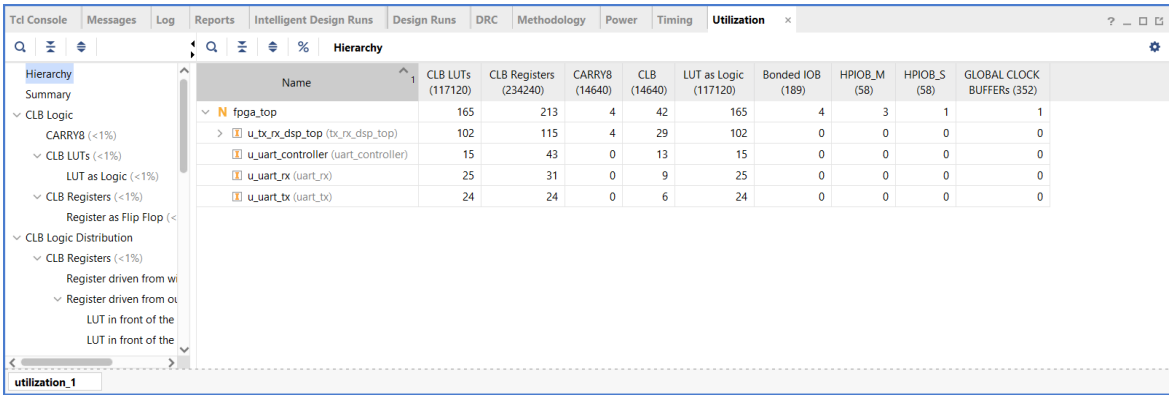


Figure 5.2. Post-implementation resource utilization reported by Vivado.

The implemented `fpga_top` uses 165 LUTs and 213 flip-flops with no BRAM, URAM, or DSP blocks. The absence of dedicated memory and DSP resources is consistent with the selected architecture, which relies primarily on XOR logic, counters, registers, comparators, and signed addition.

5.3.2 Timing Analysis

Timing slack is interpreted according to Vivado static timing analysis conventions [8]. The positive WNS and WHS values show that both setup and hold constraints are satisfied at 50 MHz.

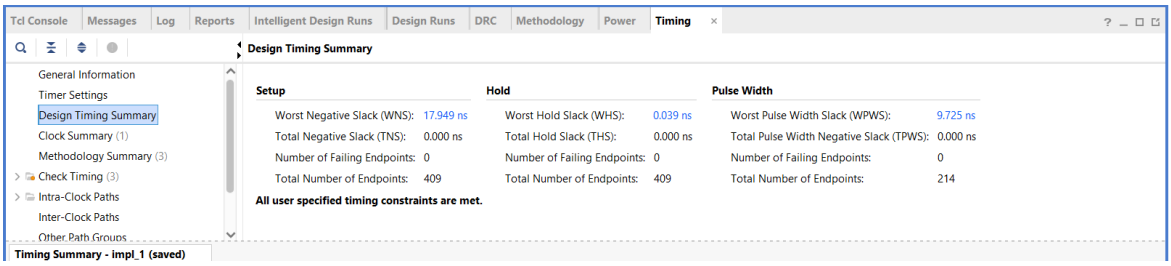


Figure 5.3. Post-implementation timing summary at the 50 MHz target clock.

From the reported setup slack, the corresponding slack-derived critical-path estimate is

$$T_{\text{path}} \approx 20 - 17.949 = 2.051 \text{ ns}. \tag{5.4}$$

The equivalent reciprocal value is approximately 488 MHz; however, this is only an estimate derived from the 50 MHz timing report. No independent frequency sweep or physical maximum-frequency measurement was performed, so it must not be interpreted as a validated F_{max}.

5.3.3 Routing and Power

All 328 routable nets are fully routed and the implementation reports zero routing errors, indicating successful completion of placement and routing for the implemented design.

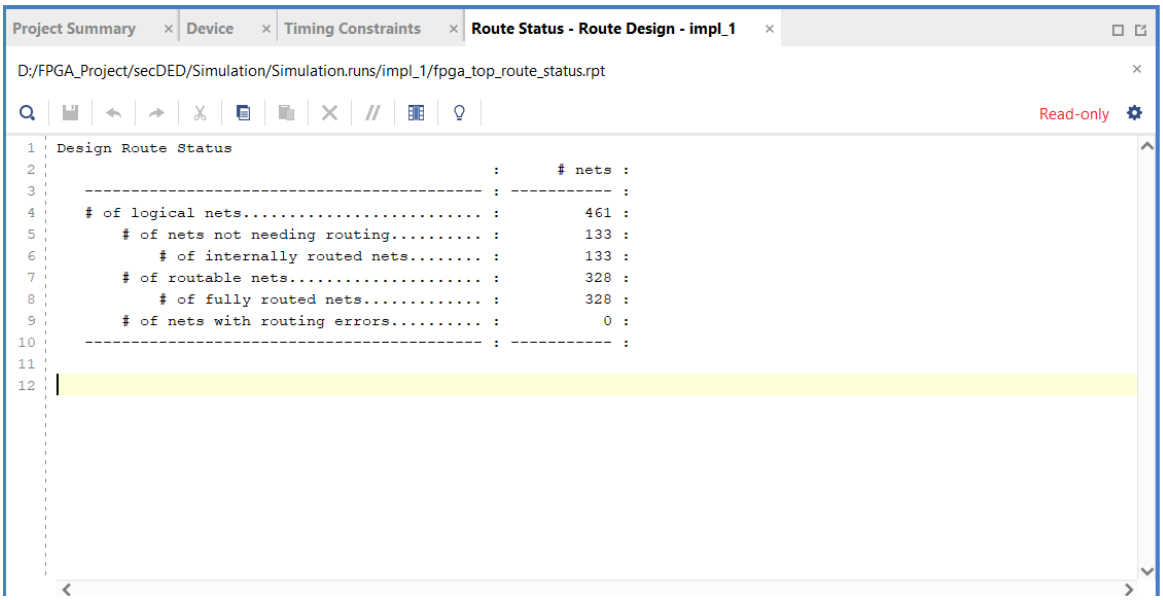


Figure 5.4. Post-implementation routing status reported by Vivado.

Vivado estimates total on-chip power at 0.289 W. The report is dominated by device static power, while the reported dynamic component is small for the current activity assumptions. Because the value is produced by post-implementation power analysis rather than direct instrumentation, it should be interpreted as a tool estimate and not as measured board-level power [9].

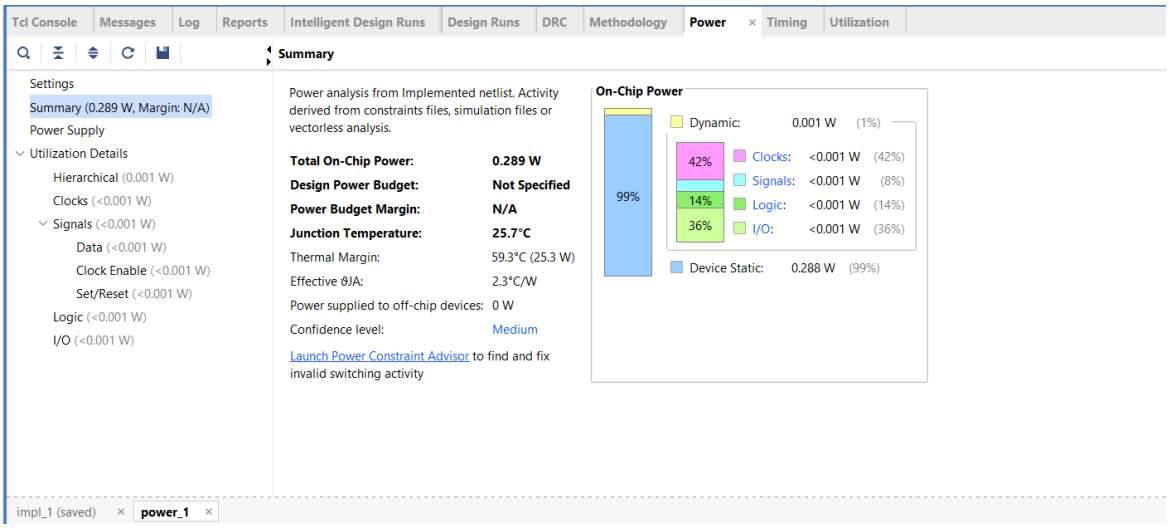


Figure 5.5. Vivado post-implementation on-chip power estimation.

5.4 Comparison with Representative FPGA Implementations

The proposed architecture is compared with representative SECDED, Hamming, and FPGA BPSK implementations. Because the cited designs use different devices, tools, and scopes, the comparison is intended to highlight architectural trade-offs rather than claim a device-independent performance advantage.

Table 5.3. Comparison with representative FPGA communication implementations.

Design	Scope	Logic	Timing	Verification
Proposed work	UART + SECDED $(8,4)$ + BPSK + dual LFSRs	165 LUTs, 213 FFs, 0 BRAM/DSP	2.051 ns estimated path	Python Golden Model, automated regression
Xilinx XAPP645 [3]	SECDED codec	Device-dependent, 0 BRAM/DSP	3.195 ns reported for a pipelined case	Hardware error injection
Tanawade and Sudhansu [10]	BPSK/QPSK modulator	79–112 LUTs, 66–80 FFs	4.386–5.301 ns	Waveform simulation
Darji and Sukhanandi [11]	BPSK modulator + demodulator	645 LUTs total	27/50 MHz operation	FPGA waveform observation
Jain et al. [12]	Hamming $(7,4)$ codec	73 core logic cells	6.500 ns	Simulation and FPGA prototype

Three observations are relevant. First, the proposed signed digital-baseband representation avoids carrier ROMs, DDS blocks, multipliers, BRAM, and DSP slices, contributing to a compact implementation relative to several carrier-oriented FPGA BPSK designs [10], [11]. Second, Extended Hamming SECDED provides explicit double-error detection in addition to single-error correction [3]. Third, the project emphasizes deterministic bit-exact verification using a Python Golden Model and self-checking regressions.

The numerical values in Table 5.3 are not directly device-independent because the cited implementations use different FPGA families, synthesis tools, coding widths, carrier representations, and integration scopes. The comparison is therefore used to highlight architectural trade-offs rather than to claim a strict cross-platform performance advantage.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project developed and verified an FPGA-oriented SECDED–BPSK digital communication system integrating error-control coding, deterministic channel disturbance, BPSK modulation and demodulation, and UART-based system communication.

A bit-exact Python Golden Model was used as the reference throughout verification. The complete DSP regression finished with zero detected errors across 384 transactions and 154,373 self-checking comparisons, while the UART end-to-end test completed 96 request/response transactions and 577 checks with zero errors.

The verified RTL design was synthesized and implemented in Vivado at a 50 MHz target clock. The final implementation uses 165 LUTs and 213 flip-flops with no BRAM, URAM, or DSP blocks, satisfies the defined setup and hold constraints, completes routing without errors, and reports an estimated total on-chip power of 0.289 W.

Overall, the project demonstrates a compact and reproducible SECDED–BPSK communication datapath in synthesizable RTL. Its main strength is the combination of a simple digital-baseband architecture with deterministic, model-based verification from individual processing stages to the complete UART-visible system.

6.2 Project Limitations

The final design has the following limitations:

- the LFSR-based noise source is deterministic rather than statistical AWGN; therefore, the project does not provide BER-versus-E_b/N_0 performance results [1];
- the configured noise magnitude does not independently reverse the polarity of the transmitted BPSK samples;
- the current implementation uses fixed SECDED $(8, 4)$ coding and fixed modulation/UART parameters;
- physical FPGA deployment and board-level measurements are outside the final project scope.

6.3 Future Work

Future work can extend the project in several directions:

- deploy the complete system on a physical FPGA platform and compare measured behavior with simulation and Vivado estimates;
- introduce a statistical AWGN channel model and evaluate BER as a function of E_b/N_0 [1];
- parameterize the modulation, sample, noise, and UART configurations for reuse in other experiments;
- investigate wider error-control codes and higher-order modulation schemes such as QPSK or QAM;
- extend the architecture with streaming interfaces, buffering, and higher-throughput data movement for future FPGA SoC integration.

REFERENCES

[1] J. G. Proakis and M. Salehi, *Digital Communications*, 5th ed. Boston: McGraw-Hill, 2008, ISBN: 9780072957167.

[2] R. W. Hamming, "Error detecting and error correcting codes," *Bell System Technical Journal*, vol. 29, no. 2, pp. 147–160, Apr. 1950. DOI: 10.1002/j.1538-7305.1950.tb00463.x Accessed: Nov. 1, 2026. [Online]. Available: https://onlinelibrary.wiley.com/doi/10.1002/j.1538-7305.1950.tb00463.x

[3] S. Tam, "Single error correction and double error detection," Xilinx, Inc., Application Note XAPP645, version 2.2, Aug. 2006, Virtex-II Pro, Virtex-4, and Virtex-5 Families. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.amd.com/v/u/en-US/xapp645

[4] S. W. Golomb, *Shift Register Sequences*, Revised. Laguna Hills, California: Aegean Park Press, 1982.

[5] AMD/Xilinx, *Axi uart lite v2.0 product guide*, PG142, version 2.0, AMD, Apr. 2017. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.amd.com/v/u/en-US/pg142-axi-uartlite

[6] AMD, *Vivado design suite user guide: Synthesis*, UG901, version 2026.1, Advanced Micro Devices, Inc., Jul. 2026. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug901-vivado-synthesis

[7] AMD, *Vivado design suite user guide: Implementation*, UG904, version 2026.1, Advanced Micro Devices, Inc., Jun. 2026. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug904-vivado-implementation

[8] AMD, *Vivado design suite user guide: Design analysis and closure techniques*, UG906, version 2026.1, Advanced Micro Devices, Inc., Jun. 2026. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug906-vivado-design-analysis

[9] AMD, *Vivado design suite user guide: Power analysis and optimization*, UG907, version 2026.1, Advanced Micro Devices, Inc., Jun. 2026. Accessed: Nov. 1, 2026. [Online]. Available: https://docs.amd.com/r/en-US/ug907-vivado-power-analysis-optimization

[10] N. Tanawade and S. Sudhansu, “Fpga implementation of digital modulation techniques bpsk and qpsk using hdl verilog,” *International Journal of Computer Applications*, vol. 165, no. 12, pp. 44–50, May 2017, ISSN: 0975-8887. DOI: 10.5120/ijca2017914106 Accessed: Nov. 1, 2026. [Online]. Available: https://www.ijcaonline.org/archives/volume165/number12/27629-2017914106/

[11] P. G. Darji and M. B. Sukhanandi, “Digital modulation/demodulation techniques on fpga,” *International Journal of Novel Research and Development*, vol. 7, no. 4, pp. 1011–1017, Apr. 2022, ISSN: 2456-4184. Accessed: Nov. 1, 2026. [Online]. Available: https://www.ijnrd.org/papers/IJNRD2204127.pdf

[12] R. Jain, P. Deshpande, and P. Shah, “Hardware acceleration of hamming code: Design of runtime reconfigurable fpga prototype,” *International Journal of Computer Applications*, vol. 96, no. 14, pp. 14–21, Jun. 2014, ISSN: 0975-8887. DOI: 10.5120/16862-6750 Accessed: Nov. 1, 2026. [Online]. Available: https://www.ijcaonline.org/archives/volume96/number14/16862-6750/